Pendo Implementation: SMS Metadata & Missing Events

Executive Summary

This document outlines the **remaining Pendo implementation work** needed for accurate user analytics. We've already implemented excellent SMS event tracking (25 events total! 🎉), but we're missing one critical piece: **SMS activity metadata in visitor profiles**.

Current State ✅

We have comprehensive event tracking including:

- SMS message processing (473 events tracked)
- Help requests via "Hey Aside" queries (54 events)
- Invite flow (11 invite commands sent)
- Search functionality (SMS and web)
- Authentication flow (197 login attempts)

The Problem ❌

SMS-only users appear "inactive" in Pendo because we only track web visits for the "Last Seen" metric. This makes our power users who primarily text look dormant.

Example:

- User A: Texts 50 times per month, visits web once → Pendo shows "inactive" ❌
- User B: Visits web daily, texts never → Pendo shows "very active" ✅

This gives us an inaccurate view of who's actually engaged with Aside.

The Solution ✨

Add SMS activity metadata to Pendo visitor profiles so we can:

1. Create accurate "Active Users" segments combining SMS + web activity
2. Segment users by platform preference (SMS-primary vs web-primary)
3. Identify at-risk users before they churn
4. Find power users for interviews

What We Already Have ✅

Current Pendo Track Events (25 total)

SMS Core Events:

- ✅ `SMS Message Received` - 473 events, 11 visitors
- ✅ `Content Added via SMS` - 192 events, 11 visitors
- ✅ `New Board Created via SMS` - 96 events, 8 visitors

Help & Support:

- ✅ `Hey_Aside_Query_Submitted` - 54 events, 9 visitors
- ✅ `Hey_Aside_Response_Sent` - 54 events, 9 visitors

Search Functionality:

- ✅ `Hybrid_Search_Query_Submitted` - 306 events, 3 visitors
- ✅ `Hybrid_Search_Results_Displayed` - 230 events, 3 visitors
- ✅ `SMS_Search_Query_Submitted` - 20 events, 6 visitors
- ✅ `SMS_Search_Results_Returned` - 20 events, 6 visitors
- ✅ `Search_Load_More_Clicked` - 4 events, 2 visitors

Invite Flow:

- ✅ `Invite Command Sent` - 11 events, 3 visitors
- ✅ `Invite Link Sent` - 11 events, 3 visitors
- ✅ `Invite Landing Viewed` - 3 events, 3 visitors
- ✅ `Invite Phone Submitted` - 1 event, 1 visitor
- ✅ `Invite Opt-In Sent` - 2 events, 2 visitors
- ✅ `Invite Conversion Completed` - 2 events, 2 visitors

Authentication:

- ✅ `Login Attempt` - 197 events, 41 visitors
- ✅ `Verification Code Sent` - 193 events, 41 visitors
- ✅ `Verification Attempt` - 193 events, 41 visitors
- ✅ `User Login Success` - 188 events, 10 visitors
- ✅ `Verification Failed` - 5 events, 3 visitors
- ✅ `Login Failed` - 4 events, 3 visitors

Other Events:

- ✅ `User Created` - 3 events, 3 visitors
- ✅ `Shared Board Created` - 3 events, 3 visitors
- ✅ `Message Created` - 20 events, 3 visitors

Great job on these! 🎉

Part 1: Priority Work - SMS Metadata in Visitor Profiles

Overview

Every time a user sends an SMS, we need to update their Pendo visitor profile with SMS activity statistics. This enables accurate "Active Users" segmentation combining SMS + web activity.

Implementation

Step 1: Helper Function to Calculate SMS Stats

javascript

```javascript
/**
 * Get SMS activity statistics for a phone number
 * @param {string} phoneNumber - User's phone number (E.164 format)
 * @returns {Object} SMS activity stats
 */
async function getSMSActivityStats(phoneNumber) {
  const now = new Date();
  const sevenDaysAgo = new Date(now - 7 * 24 * 60 * 60 * 1000);
  const thirtyDaysAgo = new Date(now - 30 * 24 * 60 * 60 * 1000);

  // Get message counts for different time periods
  const [totalMessages, last7Days, last30Days, lastMessage] = await Promise.all([
    db.messages.count({
      where: { from: phoneNumber }
    }),
    db.messages.count({
      where: {
        from: phoneNumber,
        createdAt: { gte: sevenDaysAgo }
      }
    }),
    db.messages.count({
      where: {
        from: phoneNumber,
```

```
      createdAt: { gte: thirtyDaysAgo }
    }
  }),
  db.messages.findOne({
    where: { from: phoneNumber },
    order: [['createdAt', 'DESC']]
  })
]);

// Calculate platform preference
const webVisits = await db.webSessions.count({
  where: {
    phone: phoneNumber,
    createdAt: { gte: thirtyDaysAgo }
  }
});

const totalActivity = last30Days + webVisits;
const smsPercentage = totalActivity > 0
  ? Math.round((last30Days / totalActivity) * 100)
  : 100;

let primaryPlatform;
if (smsPercentage > 70) primaryPlatform = 'sms';
else if (smsPercentage < 30) primaryPlatform = 'web';
else primaryPlatform = 'balanced';

return {
  totalMessages,
  last7Days,
  last30Days,
  lastMessageDate: lastMessage ? lastMessage.createdAt : null,
  smsPercentage,
  primaryPlatform
};
}
```

Step 2: Helper Function for Engagement Level

javascript

```
/**
 * Calculate user engagement level based on activity
 * @param {number} smsLast30Days - SMS messages in last 30 days
```

```
 * @param {number} daysSinceLastWebVisit - Days since last web visit
 * @returns {string} Engagement level
 */
function calculateEngagementLevel(smsLast30Days, daysSinceLastWebVisit) {
  // Consider both SMS and web activity
  const recentWebActivity = daysSinceLastWebVisit !== null && daysSinceLastWebVisit < 30;

  if (smsLast30Days >= 10 || (smsLast30Days >= 5 && recentWebActivity)) {
    return 'power_user';
  } else if (smsLast30Days >= 3 || recentWebActivity) {
    return 'active';
  } else if (smsLast30Days >= 1) {
    return 'casual';
  } else {
    return 'dormant';
  }
}
```

Step 3: Update Visitor Profile on Every SMS

Add this to your existing Twilio webhook handler (where you already track `SMS Message Received`):

javascript

```
app.post('/api/webhook/twilio', async (req, res) => {
  const { From, Body, MessageSid } = req.body;

  try {
    // ... your existing SMS processing logic ...
    // ... your existing pendo.track('SMS Message Received') ...

    // NEW: Get SMS activity stats
    const smsStats = await getSMSActivityStats(From);

    // NEW: Get user info
    const user = await db.users.findOne({ where: { phone: From } });

    if (user) {
      // Calculate days since signup
      const daysSinceSignup = Math.floor(
        (Date.now() - new Date(user.createdAt).getTime()) / (1000 * 60 * 60 * 24)
      );
```

```
    // Calculate days since last web visit
    const lastWebSession = await db.webSessions.findOne({
      where: { phone: From },
      order: [['createdAt', 'DESC']]
    });

    const daysSinceLastWebVisit = lastWebSession
      ? Math.floor((Date.now() - new Date(lastWebSession.createdAt).getTime()) / (1000 * 60 *
60 * 24))
      : null;

    // Get board counts
    const [privateBoards, sharedBoards] = await Promise.all([
      db.boards.count({
        where: { owner_phone: From, is_shared: false }
      }),
      db.boardMembers.count({
        where: { member_phone: From }
      })
    ]);

    // NEW: Update Pendo visitor profile with SMS metadata
    await pendo.identify({
      visitorId: From,
      accountId: 'aside', // Single account for all users
      visitor: {
        // Basic user info
        phone: From,
        firstName: user.firstName || null,
        lastName: user.lastName || null,
        email: user.email || null,

        // Signup info
        signupDate: user.createdAt.toISOString(),
        signupMethod: user.signupMethod || 'sms',
        daysSinceSignup: daysSinceSignup,

        // SMS Activity Metadata (CRITICAL FOR SEGMENTATION)
        lastSmsDate: new Date().toISOString(),
        totalSmsMessages: smsStats.totalMessages,
        smsMessagesLast7Days: smsStats.last7Days,
        smsMessagesLast30Days: smsStats.last30Days,
        daysSinceLastSms: 0, // Just sent a message!
```

```
        // Web Activity Metadata
        daysSinceLastWebVisit: daysSinceLastWebVisit,

        // Platform Preference
        primaryPlatform: smsStats.primaryPlatform,
        smsActivityPercentage: smsStats.smsPercentage,

        // User Engagement Level
        engagementLevel: calculateEngagementLevel(smsStats.last30Days,
daysSinceLastWebVisit),

        // Content Stats
        totalBoards: privateBoards + sharedBoards,
        privateBoardCount: privateBoards,
        sharedBoardCount: sharedBoards,
        hasSharedBoards: sharedBoards > 0,

        // Feature Adoption
        hasCreatedBoard: privateBoards > 0,
        hasJoinedSharedBoard: sharedBoards > 0,

        // Last updated
        profileLastUpdated: new Date().toISOString()
      }
    });
  }

  res.sendStatus(200);

 } catch (error) {
  console.error('Error processing SMS:', error);
  res.sendStatus(500);
 }
});
```

Note: This adds the `pendo.identify()` call to your existing SMS webhook without changing any existing event tracking.

Step 4: Daily Cron Job to Update daysSinceLastSms

Problem: The `daysSinceLastSms` field only updates when users send SMS. For users who haven't texted recently, this field gets stale.

Solution: Run a daily cron job to update `daysSinceLastSms` for all users.

javascript

```javascript
/**
 * Daily cron job to update daysSinceLastSms for all users
 * Run this once per day at midnight
 */
async function updateDaysSinceLastSms() {
  console.log('Starting daily SMS activity update...');

  const allUsers = await db.users.findAll({
    attributes: ['phone', 'createdAt']
  });

  for (const user of allUsers) {
    try {
      // Get last SMS message
      const lastMessage = await db.messages.findOne({
        where: { from: user.phone },
        order: [['createdAt', 'DESC']]
      });

      if (lastMessage) {
        const daysSinceLastSms = Math.floor(
          (Date.now() - new Date(lastMessage.createdAt).getTime()) / (1000 * 60 * 60 * 24)
        );

        // Update Pendo profile
        await pendo.identify({
          visitorId: user.phone,
          visitor: {
            daysSinceLastSms: daysSinceLastSms,
            profileLastUpdated: new Date().toISOString()
          }
        });
      }
    } catch (error) {
      console.error(`Error updating SMS days for ${user.phone}:`, error);
      // Continue with next user
    }
  }
```

```
  console.log('Finished daily SMS activity update');
}

// Set up cron job (using node-cron or similar)
const cron = require('node-cron');

// Run every day at midnight
cron.schedule('0 0 * * *', () => {
  updateDaysSinceLastSms();
});
```

Step 5: Backfill Historical SMS Data (One-Time)

After deploying the code above, run this ONCE to backfill SMS metadata for existing users:

javascript

```
/**
 * One-time script to backfill SMS metadata for existing users
 * Run this after deploying the pendo.identify() updates
 */
async function backfillSMSMetadata() {
  console.log('Starting SMS metadata backfill...');

  const allUsers = await db.users.findAll();
  let processed = 0;

  for (const user of allUsers) {
    try {
      // Get SMS stats
      const smsStats = await getSMSActivityStats(user.phone);

      // Calculate tenure
      const daysSinceSignup = Math.floor(
        (Date.now() - new Date(user.createdAt).getTime()) / (1000 * 60 * 60 * 24)
      );

      // Calculate days since last SMS
      const lastMessage = await db.messages.findOne({
        where: { from: user.phone },
        order: [['createdAt', 'DESC']]
      });
```

```
    const daysSinceLastSms = lastMessage
      ? Math.floor((Date.now() - new Date(lastMessage.createdAt).getTime()) / (1000 * 60 * 60 *
24))
      : null;

    // Get web visit info
    const lastWebSession = await db.webSessions.findOne({
      where: { phone: user.phone },
      order: [['createdAt', 'DESC']]
    });

    const daysSinceLastWebVisit = lastWebSession
      ? Math.floor((Date.now() - new Date(lastWebSession.createdAt).getTime()) / (1000 * 60 *
60 * 24))
      : null;

    // Get board counts
    const [privateBoards, sharedBoards] = await Promise.all([
      db.boards.count({
        where: { owner_phone: user.phone, is_shared: false }
      }),
      db.boardMembers.count({
        where: { member_phone: user.phone }
      })
    ]);

    // Update Pendo
    await pendo.identify({
      visitorId: user.phone,
      accountId: 'aside',
      visitor: {
        phone: user.phone,
        firstName: user.firstName || null,
        lastName: user.lastName || null,
        signupDate: user.createdAt.toISOString(),
        signupMethod: user.signupMethod || 'sms',
        daysSinceSignup: daysSinceSignup,

        lastSmsDate: lastMessage ? lastMessage.createdAt.toISOString() : null,
        totalSmsMessages: smsStats.totalMessages,
        smsMessagesLast7Days: smsStats.last7Days,
        smsMessagesLast30Days: smsStats.last30Days,
        daysSinceLastSms: daysSinceLastSms,
```

```
        daysSinceLastWebVisit: daysSinceLastWebVisit,

        primaryPlatform: smsStats.primaryPlatform,
        smsActivityPercentage: smsStats.smsPercentage,
        engagementLevel: calculateEngagementLevel(smsStats.last30Days,
daysSinceLastWebVisit),

        totalBoards: privateBoards + sharedBoards,
        privateBoardCount: privateBoards,
        sharedBoardCount: sharedBoards,
        hasSharedBoards: sharedBoards > 0,
        hasCreatedBoard: privateBoards > 0,
        hasJoinedSharedBoard: sharedBoards > 0,

        profileLastUpdated: new Date().toISOString()
      }
    });

    processed++;

    if (processed % 10 === 0) {
      console.log(`Processed ${processed}/${allUsers.length} users...`);
    }

    // Rate limit to avoid overwhelming Pendo API
    await new Promise(resolve => setTimeout(resolve, 100));

  } catch (error) {
    console.error(`Error backfilling ${user.phone}:`, error);
    // Continue with next user
  }
 }

 console.log(`Backfill complete! Processed ${processed} users.`);
}

// Run the backfill (comment out after running once)
// backfillSMSMetadata();
```

Testing Procedures for SMS Metadata

Test 1: Real-Time SMS Metadata Update

1. **Before test:** Check Pendo visitor profile for test phone number
    - Go to Pendo → People → Visitors → Search for phone number
    - Note current values for `totalSmsMessages`, `smsMessagesLast7Days`
2. **Action:** Send SMS from test phone to Aside
3. **Verify in Pendo (refresh after 30 seconds):**
    - Go to People → Visitors → Search for test phone number
    - Click on visitor profile
    - Check metadata fields:
        - ✅ `lastSmsDate` updated to current timestamp
        - ✅ `totalSmsMessages` incremented by 1
        - ✅ `smsMessagesLast7Days` incremented by 1
        - ✅ `daysSinceLastSms` = 0
        - ✅ `profileLastUpdated` is current timestamp

Test 2: Platform Preference Calculation

1. Check a user you know is SMS-heavy (sends lots of texts, rarely visits web):
    - ✅ `primaryPlatform: "sms"`
    - ✅ `smsActivityPercentage` > 70
2. Check a user who primarily uses web:
    - ✅ `primaryPlatform: "web"`
    - ✅ `smsActivityPercentage` < 30

Test 3: Engagement Level Calculation

1. Check a power user (10+ SMS in 30 days):
    - ✅ `engagementLevel: "power_user"`
2. Check an active user (3-9 SMS in 30 days):
    - ✅ `engagementLevel: "active"`
3. Check a casual user (1-2 SMS in 30 days):
    - ✅ `engagementLevel: "casual"`
4. Check a dormant user (0 SMS in 30 days):
    - ✅ `engagementLevel: "dormant"`

Test 4: Daily Cron Job

1. Manually trigger the `updateDaysSinceLastSms()` function
2. Check a user who hasn't texted in 5 days:
    - ✅ `daysSinceLastSms: 5`
3. Check logs for successful completion

Test 5: Historical Backfill

1. Run `backfillSMSMetadata()` script
2. Check logs for successful processing
3. Verify 3 random users in Pendo:
    - ✅ All metadata fields populated
    - ✅ `totalSmsMessages` matches database count
    - ✅ `primaryPlatform` makes sense given their activity

Success Criteria for SMS Metadata

✅ **Every SMS updates visitor profile** within 30 seconds
✅ **All metadata fields are accurate** (spot check 5 random users)
✅ **Daily cron job runs successfully** and logs completion
✅ **Historical backfill completed** for all existing users
✅ **"Active Users - Last 7 Days" segment** (created next) shows expected ~11 users
✅ **Platform preference** accurately reflects SMS vs web usage

---

Part 2: Creating Segments in Pendo (For Meredith)

Once SMS metadata is in visitor profiles, Meredith will create these segments in Pendo:

Segment 1: Active Users - Last 7 Days

Purpose: True "active users" metric combining SMS + web

Configuration:

```
Name: "Active Users - Last 7 Days"
Rules:
  daysSinceLastSms <= 7
  OR
  Last Seen <= 7 days ago
```

How to Create:

1. Go to People → Segments → Create Segment
2. Add rule: "daysSinceLastSms" "is less than or equal to" "7"
3. Click "+ OR"
4. Add rule: "Last Seen" "is within last" "7 days"

5. Save as "Active Users - Last 7 Days"

Segment 2: Active Users - Last 30 Days (MAU)

Purpose: Monthly Active Users metric

Configuration:

```
Name: "Active Users - Last 30 Days"
Rules:
  daysSinceLastSms <= 30
  OR
  Last Seen <= 30 days ago
```

Segment 3: SMS Power Users

Purpose: Users who primarily interact via SMS (high value!)

Configuration:

```
Name: "SMS Power Users"
Rules:
  smsMessagesLast30Days >= 10
  AND
  primaryPlatform = "sms"
```

Segment 4: Web-Primary Users

Purpose: Users who prefer web interface

Configuration:

```
Name: "Web-Primary Users"
Rules:
  primaryPlatform = "web"
  AND
  Last Seen <= 30 days ago
```

Segment 5: Balanced Users

Purpose: Users who use both SMS and web

Configuration:

```
Name: "Balanced Users"
Rules:
  primaryPlatform = "balanced"
  AND
  daysSinceLastSms <= 30
```

Segment 6: At-Risk Users

Purpose: Users who were active but might be churning

Configuration:

```
Name: "At-Risk Users"
Rules:
  daysSinceLastSms >= 14
  AND
  daysSinceLastSms <= 30
  AND
  Last Seen >= 14 days ago
  AND
  totalSmsMessages >= 5
```

Explanation: Users who've sent 5+ messages total but haven't been active in 14-30 days on either platform.

Segment 7: Dormant Users

Purpose: Users who've completely stopped using Aside

Configuration:

```
Name: "Dormant Users"
Rules:
  daysSinceLastSms >= 30
  AND
```

```
Last Seen >= 30 days ago
AND
daysSinceSignup >= 30
```

Segment 8: New Users (First Week)

Purpose: Track onboarding success

Configuration:

```
Name: "New Users (First Week)"
Rules:
  daysSinceSignup <= 7
```

Segment 9: Power Users for Interviews

Purpose: Find engaged users for user research

Configuration:

```
Name: "Interview Candidates"
Rules:
  engagementLevel = "power_user"
  AND
  hasCreatedBoard = true
  AND
  (hasSharedBoards = true OR sharedBoardCount >= 1)
  AND
  daysSinceSignup >= 14
```

Explanation: Power users who've been around at least 2 weeks, created boards, and collaborate (likely to have valuable insights).

Part 3: Optional Enhancement - Add Properties to Existing Events

These are **nice-to-haves** that enhance existing events with more context:

Enhancement 1: Add Properties to `Hey_Aside_Query_Submitted`

Current event: Tracks that user asked a question
Enhancement: Add properties to categorize what they're asking about

Code change in SMS webhook where you already track this event:

javascript
```
// EXISTING: You already have this event
await pendo.track('Hey_Aside_Query_Submitted', {
  user_id: From,
  // ... your existing properties ...
});

// ENHANCEMENT: Add these additional properties
await pendo.track('Hey_Aside_Query_Submitted', {
  user_id: From,
  // ... your existing properties ...

  // NEW PROPERTIES:
  question_type: detectQuestionType(Body), // Categorize the question
  user_tenure_days: calculateTenure(From), // How long they've been a user
  is_repeat_question: await checkIfRepeatQuestion(From, questionType) // Asked before?
});
```

Helper function:

javascript
```
function detectQuestionType(messageText) {
  const text = messageText.toLowerCase();

  const questionTypes = {
    'how does aside work': 'how_does_aside_work',
    'how do i create a board': 'create_board',
    'how do i add': 'add_to_board',
    'create a shared board': 'create_shared_board',
    'invite someone': 'invite_to_board',
    'my login': 'dashboard_link',
    'my dashboard': 'dashboard_link',
    'what boards': 'board_list',
    'move': 'move_item',
    'delete an item': 'delete_item',
    'delete a board': 'delete_board'
  };
```

```
  for (const [phrase, type] of Object.entries(questionTypes)) {
    if (text.includes(phrase)) {
      return type;
    }
  }

  return 'other';
}
```

Why this matters: You'll be able to see "40% of help requests are about creating boards" → indicates onboarding gap

---

Part 4: Future Events (When You Send These Messages)

These events should be added when you implement the corresponding SMS touchpoints:

Event: Anniversary Message Sent

When: You send 1-month, 3-month, or 6-month anniversary messages

Code:

javascript
```
// In your scheduled job that sends anniversary messages
await pendo.track('Anniversary_Message_Sent', {
  user_id: phoneNumber,
  milestone: '1_month' | '3_month' | '6_month',
  user_active_status: recentSmsCount > 0 ? 'active' : 'dormant',
  days_since_signup: daysSinceSignup,
  recent_sms_count_30d: recentSmsCount,
  timestamp: new Date().toISOString()
});
```

Event: Board List Request Sent

When: User texts asking "what boards do I have?"

Code:

```javascript
// In SMS webhook when detecting board list requests
if (messageText.includes('what boards') || messageText.includes('my boards')) {
  await pendo.track('Board_List_Request_Sent', {
    user_id: From,
    total_boards: boardCount,
    user_tenure_days: calculateTenure(From),
    timestamp: new Date().toISOString()
  });
}
```

Event: Dashboard Link Request Sent

When: User texts asking for their login/dashboard link

Code:

```javascript
// In SMS webhook when detecting dashboard link requests
if (messageText.includes('my login') || messageText.includes('my dashboard')) {
  await pendo.track('Dashboard_Link_Request_Sent', {
    user_id: From,
    user_tenure_days: calculateTenure(From),
    days_since_last_web_visit: daysSinceLastWebVisit,
    timestamp: new Date().toISOString()
  });
}
```

Implementation Timeline

Phase 1: SMS Metadata Foundation (2-3 days)

Priority: CRITICAL

- Day 1: Implement `getSMSActivityStats()` and `calculateEngagementLevel()` helpers
- Day 1: Add `pendo.identify()` calls to SMS webhook
- Day 2: Implement daily cron job for `daysSinceLastSms`
- Day 2: Test with SMS sends, verify profile updates in Pendo

- Day 3: Run backfill script for existing users
- Day 3: Verify all users have metadata populated

Phase 2: Create Segments (1 day)

Priority: HIGH Owner: Meredith

- Day 1: Create all 9 segments in Pendo UI
- Day 1: Verify segment counts look correct
- Day 1: Set up initial analytics dashboards

Phase 3: Optional Enhancements (1-2 days)

Priority: MEDIUM Can wait until after launch

- Add `question_type` property to `Hey_Aside_Query_Submitted`
- Add anniversary message tracking (when implemented)
- Add board list request tracking (when implemented)
- Add dashboard link request tracking (when implemented)

Total Timeline: 3-4 days for critical path (Phases 1-2)

Complete List of Visitor Metadata Fields

Here's every field we're adding to Pendo visitor profiles:

Basic User Info

- `phone` - E.164 format phone number
- `firstName` - User's first name (if provided)
- `lastName` - User's last name (if provided)
- `email` - User's email (if provided)

Signup Info

- `signupDate` - ISO timestamp of account creation
- `signupMethod` - "sms" or "web"
- `daysSinceSignup` - Days since they joined Aside

SMS Activity (CRITICAL)

- `lastSmsDate` - ISO timestamp of most recent SMS
- `totalSmsMessages` - Lifetime SMS message count
- `smsMessagesLast7Days` - SMS count in last 7 days
- `smsMessagesLast30Days` - SMS count in last 30 days
- `daysSinceLastSms` - Days since most recent SMS

Web Activity

- `daysSinceLastWebVisit` - Days since last web app visit
- (Note: Pendo's built-in "Last Seen" tracks web visits automatically)

Platform Preference

- `primaryPlatform` - "sms", "web", or "balanced"
- `smsActivityPercentage` - % of total activity that's SMS
- `engagementLevel` - "power_user", "active", "casual", "dormant"

Content Stats

- `totalBoards` - Total number of boards (private + shared)
- `privateBoardCount` - Number of private boards owned
- `sharedBoardCount` - Number of shared boards they're in
- `hasSharedBoards` - Boolean: true if in any shared boards
- `hasCreatedBoard` - Boolean: true if created any board
- `hasJoinedSharedBoard` - Boolean: true if joined any shared board

System

- `profileLastUpdated` - ISO timestamp of last metadata update

Questions for Meredith

1. **Engagement level thresholds:** Do these make sense?
    - Power user: 10+ SMS/month OR 5+ SMS + recent web
    - Active: 3+ SMS/month OR recent web visit
    - Casual: 1-2 SMS/month
    - Dormant: 0 SMS in 30 days
2. **Cron job timing:** Is midnight okay for the daily update, or prefer a different time?
3. **Rate limiting:** Should we add delays between Pendo API calls in the backfill to avoid hitting rate limits? (Currently set to 100ms between calls)

4. **Anniversary messages:** Once we implement these, should we send to all users or just active users?

Summary: What This Solves

Before This Implementation ❌

- SMS-only users appear "inactive" in Pendo
- Can't accurately count "active users"
- Don't know who's SMS-primary vs web-primary
- Can't identify at-risk users
- Power users who primarily text look dormant

After This Implementation ✅

- **Accurate "Active Users" metric** combining SMS + web activity
- **Platform preference segmentation** (SMS-primary, web-primary, balanced)
- **At-risk user identification** for proactive retention
- **Power user discovery** for interviews and user research
- **True engagement tracking** across all touchpoints